AGENDA 7

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

TIAGO ANTONIO DA SILVA

Revisão Técnica:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

Você já se perguntou como um aplicativo de clima sabe a temperatura da sua cidade ou como um App de delivery encontra restaurantes próximos de você?

Isso acontece graças ao consumo de APIs externas, também conhecidas como Web Services. Nesta agenda, vamos aprender a integrar o aplicativo a essas fontes de dados externas, ampliando as possibilidades e funcionalidades do que podemos desenvolver.

MERGULHANDO NO TEMA

API (Application Programming Interface), ou Interface de Programação de Aplicações, funciona como uma ponte entre diferentes sistemas ou aplicações, permitindo que eles conversem entre si de forma padronizada.

Vamos a um exemplo!

Imagine que você vai ao restaurante e faz um pedido ao garçom. O garçom leva seu pedido até a cozinha e depois retorna com o prato. Neste exemplo:

- **Você** é o aplicativo.
- **A cozinha** é o servidor onde os dados estão armazenados.
- **O garçom é a API**: ele leva e traz informações.

APIs funcionam exatamente assim: elas são a ponte entre o seu aplicativo e um serviço externo.

Saiba mais sobre definição de API:
Disponível em: *https://www.youtube.com/watch?v=vGuqKIRWosk*
Fonte: API // Dicionário do Programador. Acessado em 27/03/2025

Web Services

Web services são sistemas que permitem a comunicação e a troca de dados entre diferentes aplicações por meio da internet ou de redes privadas. Eles seguem padrões e protocolos abertos para garantir a interoperabilidade entre sistemas desenvolvidos em diferentes linguagens de programação e plataformas. A principal vantagem do uso de Web Services é a capacidade de integrar sistemas distintos, tornando possível a comunicação entre aplicações de diferentes empresas, dispositivos ou servidores.

Os Web Services funcionam por meio de protocolos de comunicação como HTTP e utilizam formatos padronizados para a troca de informações, como XML e JSON. Dois dos principais tipos de Web Services são

o SOAP (Simple Object Access Protocol) e REST (Representational State Transfer).

O SOAP é baseado em XML e segue um conjunto rigoroso de regras, sendo amplamente utilizado em ambientes corporativos que exigem segurança e confiabilidade. Já o REST é mais leve e flexível, utilizando operações HTTP como GET, POST, PUT e DELETE para manipulação de recursos.

Na prática, os Web Services são amplamente utilizados em aplicações web, sistemas empresariais e dispositivos móveis, permitindo por exemplo, que um site de e-commerce consulte informações de pagamento de um serviço externo, ou um aplicativo de clima obtenha previsões meteorológicas de uma API pública ou que uma plataforma de redes sociais compartilhe dados com outras aplicações. Essa capacidade de integração facilita o desenvolvimento de soluções escaláveis e eficientes.

Com o avanço da computação em nuvem e das arquiteturas baseadas em Micro Serviços, os Web Services se tornaram ainda mais essenciais. Empresas modernas dependem de APIs baseadas em Web Services para conectar sistemas internos, parceiros e clientes. Assim, a utilização dessas tecnologias melhora a eficiência, a automação de processos e a experiência do usuário.

Acessando Web Services com HttpClient

A classe HttpClient do .NET é uma ferramenta poderosa para realizar requisições HTTP e interagir com Web Services. Ela permite que aplicações .NET enviem e recebam dados de servidores web, utilizando protocolos como HTTP e HTTPS. Essa classe faz parte do namespace System.Net.Http e é projetada para ser eficiente em termos de desempenho e reutilização de conexões, sendo recomendada para consumo de APIs RESTful e comunicação entre sistemas distribuídos.

O HttpClient facilita a realização de operações como GET, POST, PUT e DELETE, que são comuns ao interagir com web services baseados em REST.

Ele suporta o envio de cabeçalhos personalizados, autenticação, serialização de objetos para JSON ou XML e manipulação de respostas assíncronas. Como essa classe é baseada no modelo assíncrono do .NET, ela ajuda a evitar bloqueios de threads (travar o app), tornando as chamadas de rede mais eficientes e responsivas.

Ao consumir um Web Service com HttpClient, o desenvolvedor pode utilizá-lo para obter dados de uma API externa, enviar informações para um servidor ou integrar diferentes partes de uma aplicação distribuída. Por exemplo, em um sistema de e-commerce, um serviço de pagamento pode ser acessado por meio do HttpClient, enviando detalhes da transação e recebendo uma resposta sobre a aprovação do pagamento. Esse tipo de integração é essencial para arquiteturas modernas, onde diferentes serviços se comunicam por meio de APIs.

Para obter o máximo de desempenho e evitar problemas como exaustão de conexões, é recomendável

instanciar HttpClient de forma reutilizável, como um singleton. Essa abordagem melhora a eficiência, evitando a criação excessiva de conexões e garantindo uma comunicação mais estável com Web Services. Assim, o HttpClient se torna uma peça fundamental na integração de aplicações modernas com APIs e sistemas externos.

Classe HttpResponseMessage

A classe HttpResponseMessage do .NET faz parte do namespace System.Net.Http e é usada para representar a resposta de uma requisição HTTP feita com a classe HttpClient. Ela encapsula informações importantes sobre a resposta recebida de um servidor, como o código de status HTTP, os cabeçalhos, o conteúdo da resposta e mensagens de erro, quando aplicável.

As principais propriedades são:

- **StatusCode**: Representa o código de status HTTP retornado pelo servidor (por exemplo, 200 OK, 404 Not Found, 500 Internal Server Error).
- **ReasonPhrase**: Uma mensagem descritiva do status da resposta, como "OK" ou "Not Found".
- **Content**: Contém o corpo da resposta, que pode ser lido como JSON, string ou outro formato.
- **Headers**: Coleção de cabeçalhos HTTP retornados pelo servidor.
- **IsSuccessStatusCode**: Retorna true se a resposta indicar sucesso (códigos de status entre 200 e 299).

Quando uma aplicação consome uma API RESTful, ela utiliza HttpClient para fazer requisições HTTP e recebe um objeto HttpResponseMessage contendo a resposta do servidor. O desenvolvedor pode então analisar essa resposta para decidir o próximo passo, como tratar erros, processar dados ou realizar novas chamadas. Para evitar problemas de desempenho e gerenciamento inadequado de conexões, é recomendável que o HttpResponseMessage seja descartado corretamente com using. Essa prática ajuda a liberar recursos e evitar possíveis vazamentos de memória.

A HttpResponseMessage é essencial para o consumo eficiente de APIs e web services no .NET, permitindo que desenvolvedores lidem com respostas HTTP de forma estruturada e robusta.

JSON

JSON (JavaScript Object Notation) é um formato leve para a troca de dados, amplamente utilizado em aplicações web e APIs. Ele é baseado em uma estrutura de pares chave-valor, semelhante a objetos em JavaScript, tornando-o fácil de ler e escrever tanto para humanos quanto para máquinas. Sua simplicidade e compatibilidade com diversas linguagens de programação o tornaram um padrão popular para comunicação

entre sistemas. Vamos a um exemplo:

```
{
  "nome": "João",
  "idade": 30,
  "cidade": "São Paulo"
}
```

No exemplo apresentado, temos um objeto com três chaves (nome, idade e cidade), cada uma associada a um valor correspondente.

JSON é frequentemente utilizado em Web Services para serializar e transmitir dados entre servidores e clientes. APIs RESTful, por exemplo, retornam respostas em JSON, permitindo que aplicativos web e móveis consumam e processem informações de maneira eficiente. Além disso, bibliotecas em linguagens além do C#, como Python, Java e JavaScript oferecem suporte nativo para manipulação de JSON, facilitando sua integração em projetos.

Uma das vantagens do JSON em relação a outros formatos, como XML, é sua menor verbosidade e melhor desempenho na serialização e desserialização. O JSON também é amplamente utilizado em bancos de dados NoSQL, como o MongoDB, onde os documentos são armazenados em um formato semelhante ao JSON, permitindo consultas e manipulações flexíveis.

O pacote Newtonsoft.Json, também conhecido como Json.NET, é uma das bibliotecas mais populares para manipulação de JSON no .NET. Ele é amplamente utilizado para serialização (converter objetos .NET em JSON) e desserialização (converter JSON em objetos .NET), além de oferecer recursos avançados para manipulação de dados estruturados.

Serialização e Desserialização Simples

Permite converter objetos C# para JSON e vice-versa de maneira rápida e eficiente.

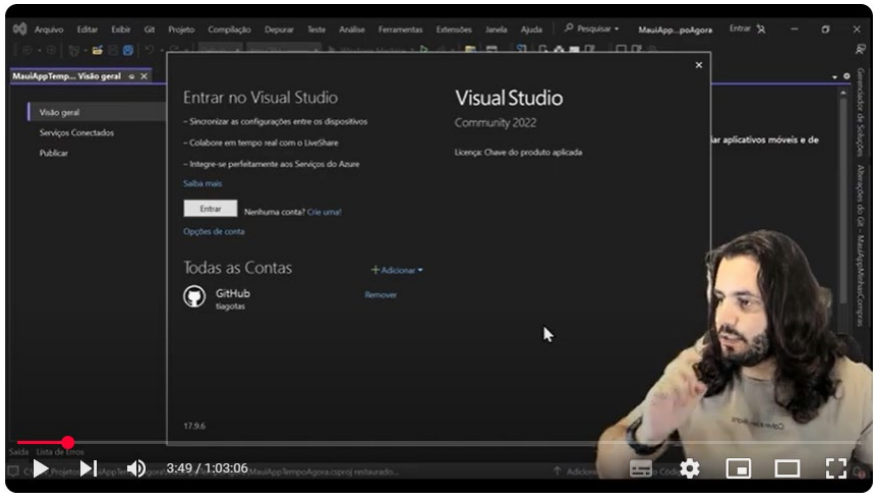
Veja um exemplo de serialização, ou seja, transformar um objeto em C# para uma string JSON:

```
using Newtonsoft.Json;

var pessoa = new { Nome = "João", Idade = 30 };
string json = JsonConvert.SerializeObject(pessoa);
Console.WriteLine(json); // {"Nome":"João","Idade":30}
```

Vejamos um exemplo de desserialização, o processo contrário de JSON para C#:

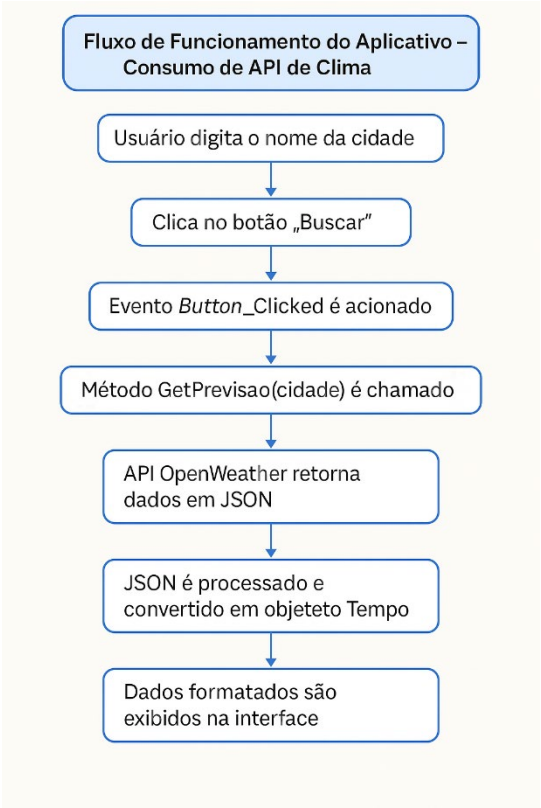
```
using Newtonsoft.Json;

string json = "{\"Nome\":\"Maria\",\"Idade\":25}";
var pessoa = JsonConvert.DeserializeObject<dynamic>(json);
Console.WriteLine(pessoa.Nome); // Maria
```

Assista o vídeo apresentando o passo a passo para criação do aplicativo:



Disponível em: https://www.youtube.com/watch?v=t0QcPBYvYRw

Vamos compreender o fluxo do App apresentado no projeto:



CÓDIGO-FONTE DA AGENDA:

https://github.com/tiagotas/MauiAppTempoAgora/tree/d6f7d35ec23a99a97ed79c835ab55217d38eac82